

EAST WEST UNIVERSITY



Department of Computer Science & Engineering

Term Project

Theoretical vs. Empirical Analysis of Algorithms

Course Title: Introduction to Algorithms

Course Code: CSE 246

Dr. Hasan Mahmood Aminul Islam

Assistant Professor, Dept of CSE

Email: hasan.mahmood@ewubd.edu

Objective

The primary objective of this assignment is to bridge the gap between the theoretical analysis of algorithms (using Big-O notation) and their practical performance. You will analyze, implement, and benchmark a set of fundamental algorithms to observe how their execution time scales with input size.

Task Overview

For each of the 10 algorithms listed below, you are required to perform the following steps:

1. Theoretical Analysis:

- Determine and state the time complexity (Big-O notation) in the worst-case scenario.
- Determine and state the space complexity (Big-O notation).
- Write a brief, clear justification (1-3 sentences) for both complexities. Explain why the algorithm exhibits this behavior.

2. Implementation & Empirical Analysis:

- Implement the algorithm in a programming language of your choice (Python, Java, or C++ are recommended).
- Generate appropriate input data for the following sizes of n : **1, 10, 100, 1000, 10000**. (See the Important Notes section for exceptions).
- Measure the execution time of your implementation for each input size. To get stable results, you may need to run the algorithm multiple times for a given n and take the average time.
- Organize your measured timings in a clearly formatted table.

3. Visualization:

- Create a single plot/graph that shows the performance of all algorithms.
- The x-axis should represent the Input Size (n).
- The y-axis should represent the Time Taken (in milliseconds or nanoseconds).
- **Crucially, use a logarithmic scale for the y-axis** to ensure that both very fast and very slow algorithms are visible on the same graph.
- Clearly label your axes and provide a legend to distinguish between the algorithms.

List of Algorithms to Analyze

1. **Linear Search:** Searching for an element in an unsorted array.
2. **Binary Search:** Searching for an element in a sorted array.

3. **Merge Sort:** A divide-and-conquer sorting algorithm.
4. **Quick Sort:** A divide-and-conquer sorting algorithm. (Analyze its average-case complexity, but be aware of the worst-case).
5. **Recursive Fibonacci:** The classic recursive implementation to find the n -th Fibonacci number. Note the exponential complexity.
6. **0/1 Knapsack (Dynamic Programming):** Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.
7. **Breadth-First Search (BFS):** Traversing a graph.
8. **Dijkstra's Algorithm:** Finding the shortest path from a single source vertex to all other vertices in a weighted graph with non-negative edge weights.
9. **Prim's Algorithm:** Finding a Minimum Spanning Tree (MST) for a weighted undirected graph.
10. **Floyd-Warshall Algorithm:** Finding the shortest paths between all pairs of vertices in a weighted graph.

Important Notes on Implementation and Data Generation

- **Input Size n :** For graph-based algorithms, n refers to the number of vertices (V).
- **Data Generation:**
 - **Sorting/Searching:** For sorting, use an array of random integers. For searching, search for an element not present (worst case).
 - **Graph Algorithms (BFS, Dijkstra, Prim's, Floyd-Warshall):** Generate a random dense graph with n vertices and edge weights. A dense graph means the number of edges E is proportional to n^2 .
 - **0/1 Knapsack:** The input size n is the number of items. Generate n items with random weights and values. The knapsack capacity W should also scale with n ; a good choice is $W=50 \times n$.
- **Timing:** Use high-precision timers available in your language's standard library (e.g., `time.perf_counter()` in Python, `System.nanoTime()` in Java, `<chrono>` library in C++).
- **Exceptions for Input Size (n):**
 - **Recursive Fibonacci:** The complexity of this algorithm is exponential ($O(2^n)$). It will be infeasibly slow for large n . You should test it for smaller values, for example, $n=1, 5, 10, 15, 20, 25, 30$. **Do NOT attempt to run it for $n=1000$.** In your report, explain why this is the case.
 - **Floyd-Warshall Algorithm:** The complexity of this algorithm is $O(n^3)$. This will be very slow for large n . If $n=1000$ or $n=10000$ takes too long to execute (e.g., more than a few minutes), you may cap your testing at a smaller size (e.g., $n=500$). Make a note of this in your report.

Submission Guidelines

You must submit a single PDF report containing the following sections:

1. **Introduction:** A brief overview of the assignment's purpose.
2. **Algorithm Analysis:** For each of the 10 algorithms, create a subsection that includes:
 - Algorithm Name
 - Time Complexity & Justification
 - Space Complexity & Justification
 - A table of your empirically measured execution times vs. input size n .
3. **Performance Graph:**
 - The generated graph visualizing the execution times for all algorithms.
4. **Conclusion & Discussion:**
 - Discuss your findings. Does the empirical data from your graph generally match the theoretical Big-O complexities? For example, is the curve for Floyd-Warshall ($O(n^3)$) significantly steeper than the one for Merge Sort ($O(n \log n)$)?
 - Explain any significant discrepancies or observations. (e.g., "For small n , Algorithm X was faster than Algorithm Y because of lower constant factors...")
 - Reflect on the practical limitations of algorithms with high polynomial or exponential complexity.
5. **Appendix:**
 - Your complete, well-commented source code.

Grading Rubric (Total 100 Points)

- **Theoretical Analysis (40 points):** 4 points for each algorithm (2 for time complexity/justification, 2 for space complexity/justification).
- **Implementation Correctness (20 points):** Code is well-structured, correct, and implements the algorithms as specified.
- **Empirical Data Collection (15 points):** The data table is present, complete, and accurately reflects the measured performance.
- **Visualization (15 points):** The graph is clear, well-labeled, uses a logarithmic scale correctly, and accurately represents the collected data.
- **Conclusion and Report Quality (10 points):** The final report is well-written, and the conclusion provides insightful analysis connecting theory with practice.

Bonus: Do the same for Travelling salesman problems (TSP).

